
Initiation à Maven

Table des matières

1. Nathaniel Vaur Henel
2. Pourquoi utiliser un outil comme Maven ?
3. Qu’est-ce que Maven ?
4. Installation et configuration
5. Structure d’un projet Maven
6. Le fichier pom.xml
7. Commandes Maven essentielles
8. Création d’un projet Maven simple
9. Utiliser des dépendances externes
10. Avantages et bonnes pratiques
11. Pour aller plus loin



© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**



Nathaniel Vaur Henel

L'auteur

Auteur

 Nathaniel Vaur Henel 🤘

- 👨‍🏫 Formateur
- 👨‍💻 Tech leader
- 🧉 Expert Java
- 💻 20 ans d'expérience en développement, principalement en Java
- 📜 Créateur de contenus pour dévs : dev-sensei.digicrafters.fr
- Discord : `@nathanielvaurhenel`



framagit.org/nathvh/me



dev-sensei.digicrafters.fr



linkedin.com/in/nathaniel-vaur-henel

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**


Pourquoi utiliser un outil comme Maven ?

Introduction au besoin d'un outil de gestion de projet Java

La gestion de projets Java peut rapidement devenir complexe sans outil adapté.
Ce chapitre explique pourquoi Maven s'impose comme solution incontournable dans un contexte professionnel ou personnel.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Problèmes avec la gestion manuelle des dépendances

- Conflits de versions 👄 : plusieurs bibliothèques peuvent dépendre de versions différentes d'une même librairie.
- Duplication inutile 🔁 : chaque développeur télécharge et ajoute manuellement les fichiers `.jar`.
- Difficulté de mise à jour 📦 : aucune centralisation, donc pas de gestion automatisée.
- Risques d'incompatibilité ❌ : erreurs d'exécution fréquentes si les dépendances ne sont pas cohérentes.

Reproductibilité et portabilité des projets

- Maven permet d'assurer la reproductibilité des builds 🚀 grâce à un fichier de configuration unique : `pom.xml`.
- Le projet est indépendant du poste de travail : plus besoin de transférer les `.jar` manuellement.
- Les étapes de build sont automatisées (test, compilation, packaging) ➕ portables.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Présentation rapide des outils similaires

- **Ant** : outil plus ancien, basé sur des scripts XML, sans gestion native des dépendances.
- **Gradle** : outil moderne et flexible, utilise Groovy/Kotlin DSL, plus rapide pour les projets complexes.
- **Pourquoi Maven ?** 🎯
  - Standard de fait dans l'écosystème Java.
  - Documentation abondante 📚.
  - Support natif dans les IDE (Eclipse, IntelliJ) ✔️.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**


Qu’est-ce que Maven ?

Définition de Maven

Maven est un outil de gestion et d'automatisation de projet Java. Il permet de centraliser et standardiser la façon dont un projet est compilé, testé, empaqueté et déployé.

Il repose sur le concept de **Project Object Model (POM)**, représenté par le fichier `pom.xml`.

Fonctionnalités principales : gestion des dépendances

Maven télécharge automatiquement les bibliothèques nécessaires à un projet depuis des dépôts centralisés (comme Maven Central).

Exemple :

```
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
</dependency>
```

Cela évite de stocker manuellement des `.jar` et garantit des versions cohérentes.

Fonctionnalités principales : compilation, tests, packaging

Maven intègre un cycle de vie standardisé :

- **clean** : nettoie les fichiers générés
- **compile** : compile le code source
- **test** : exécute les tests unitaires
- **package** : génère un `.jar` ou `.war`
- **install** : installe dans le dépôt local

Ces phases sont invoquées avec des commandes simples comme `mvn package`.

Fonctionnalités principales : documentation et rapports

Maven permet aussi de générer :

- Une documentation de projet (`mvn site`) même si c'est peu utilisé
- Des rapports de tests, de couverture de code, de dépendances

Ces outils sont utiles en entreprise pour assurer la qualité et la traçabilité.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Structure standard d’un projet Maven

Maven impose une convention d’organisation des fichiers :

```
project-root/
├── pom.xml
└── src/
    ├── main/java/       # code source
    ├── main/resources/  # ressources (config, fichiers statiques)
    ├── test/java/       # tests unitaires
    └── test/resources/  # ressources de test
```

Cela favorise la standardisation entre projets, facilitant la maintenance.

Installation et configuration



Installer Maven sur différents systèmes

🖥️ Windows

- Télécharger l'archive ZIP depuis maven.apache.org
- Extraire dans `C:\Program Files\Apache\maven` ou pas…

🐧 Linux (via package manager)

```
sudo apt install maven      # Debian/Ubuntu
sudo dnf install maven      # Fedora
```

🍎 macOS (via Homebrew)

```
brew install maven
```

Vérification de l'installation avec `mvn -v`

Après l'installation, il est essentiel de vérifier que Maven est accessible en ligne de commande :

```
mvn -v
```

Exemple de sortie attendue :

```
Apache Maven 3.8.6
Java version: 17.0.2
OS name: "linux"
```

Cela confirme que Maven est correctement installé et lié au JDK.

Variables d'environnement à configurer

Trois variables importantes doivent être configurées :

`JAVA_HOME`

Doit pointer vers le dossier d'installation du JDK.

`MAVEN_HOME`

Optionnelle, mais peut pointer vers le répertoire Maven.

`PATH`

Doit inclure `bin` de Maven et du JDK :

```
export PATH=$PATH:$JAVA_HOME/bin:$MAVEN_HOME/bin
```

Ces variables permettent à Maven de fonctionner sur tous les systèmes.

Maven intégré dans IntelliJ IDEA

L'IDE **IntelliJ IDEA** intègre Maven de façon native :

- Aucun téléchargement ni configuration manuelle n'est nécessaire.
- Il suffit de créer un projet **Maven** depuis l'assistant de projet.
- IntelliJ détecte et télécharge automatiquement les dépendances à partir du `pom.xml`.

Cela facilite la prise en main pour les débutants et accélère le développement.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**


Structure d’un projet Maven

Arborescence type d'un projet Maven

Voici la structure standard d'un projet Maven :

```
mon-projet/
├── pom.xml
└── src/
    ├── main/
    │   ├── java/        # Code source
    │   ├── resources/   # Ressources (fichiers de configuration, etc.)
    └── test/
        ├── java/        # Tests unitaires
        └── resources/   # Ressources de test
```

Chaque projet Maven suit cette organisation par défaut, ce qui facilite la compréhension, la maintenance et l'intégration dans des outils automatisés.

Rôle de chaque dossier dans la structure

- `pom.xml` : fichier central qui définit les dépendances, plugins, configurations du projet 📄
- `src/main/java/` : contient le code source principal de l'application 💻
- `src/test/java/` : contient les classes de test (JUnit ou autre) 🧪

Cette organisation sépare clairement les responsabilités dans le projet.

Convention over configuration

Maven repose sur le principe de **convention over configuration** :

- Inutile de spécifier les chemins standards dans `pom.xml` ✅
- Si l'on respecte l'arborescence type, Maven "sait quoi faire"
- Cela réduit les fichiers de configuration, tout en gardant une grande flexibilité via les plugins

Exemple : aucun paramètre nécessaire pour compiler le contenu de `src/main/java`

Le fichier pom.xml



Structure de base du pom.xml (1/2)

Le fichier `pom.xml` est le cœur d'un projet Maven. Il contient les métadonnées et la configuration du projet.

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="h
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.exemple</groupId>
    <artifactId>mon-projet</artifactId>
    <version>1.0.0</version>
    <packaging>jar</packaging>

    <properties>...</properties>

    <dependencies>
        <dependency>...</dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>...</plugin>
        </plugins>
    </build>
</project>
```

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Structure de base du pom.xml (2/2)

- `groupId` : identifiant de l'organisation ou du projet
- `artifactId` : nom unique du module ou de l'application
- `version` : version du projet
- `packaging` : type d'artefact généré (ex : `jar`, `war`)
- `properties` : permet de centraliser des paramètres réutilisables (`maven.compiler.source`, etc.)
- `dependencies` : liste des bibliothèques externes nécessaires au projet
- `build` : configuration de la phase de build
  - `plugins` : liste des plugins utilisés pour enrichir le cycle de vie (ex: compiler, test, jar, site…)

👉 Le trio `groupId`, `artifactId` et `version` permet d'**identifier de manière unique** un projet dans un dépôt Maven (comme Maven Central). Cela garantit qu'il n'y a **aucune ambiguïté** lorsqu'un projet dépend d'un autre.

Ajouter une dépendance dans le pom.xml

Pour inclure une bibliothèque externe, on utilise une balise `<dependency>` :

Exemple avec **JUnit** :

```
<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.10.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Autre exemple : **Gson** pour le traitement JSON :

```
<dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.10.1</version>
</dependency>
```

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Configuration de plugins simples dans le pom.xml

Les **plugins Maven** permettent d'ajouter des comportements personnalisés au cycle de vie. Exemple de plugin pour compiler :

```
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.10.1</version>
            <configuration>
                <source>17</source>
                <target>17</target>
            </configuration>
        </plugin>
    </plugins>
</build>
```

Ce plugin configure la version du JDK utilisée lors de la compilation.


Commandes Maven essentielles

Compiler le code avec `mvn compile`

Cette commande compile le code source du projet à partir de `src/main/java` :

```
mvn compile
```

Elle génère les fichiers `.class` dans le dossier `target/classes`.

Créer un fichier JAR avec `mvn package`

Cette commande compile le projet, lance les tests, puis empaquette l'application en `.jar` :

```
mvn package
```

Le fichier `.jar` généré est disponible dans le répertoire `target/`.

Exécuter les tests avec `mvn test`

Cette commande exécute les tests unitaires définis dans `src/test/java` :

```
mvn test
```

Par défaut, Maven utilise le plugin `maven-surefire-plugin` pour lancer les tests.

Nettoyer le projet avec `mvn clean`

Supprime tous les fichiers générés lors des builds précédents :

```
mvn clean
```

Efface le dossier `target/`, permettant de repartir sur une base propre.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Installer dans le dépôt local avec `mvn install`

Cette commande compile, teste, empaquette, puis installe l'artefact dans le dépôt Maven local (`~/.m2/repository`) :

```
mvn install
```

Utile pour rendre le projet disponible à d'autres modules locaux.

Utilisation de Maven dans IntelliJ IDEA

Dans **IntelliJ IDEA**, Maven est intégré et les actions sont déclenchées via l'interface graphique ou la console intégrée :

- Lancement des commandes via les boutons *Lifecycle* (clean, compile, package, etc.) dans le panneau *Maven*.
- Le terminal IntelliJ peut parfois ne pas reconnaître `mvn` s'il n'est pas configuré dans l'environnement système.
- IntelliJ utilise sa propre configuration Maven embarquée si aucune autre n'est détectée.

💡 Il est recommandé de vérifier les paramètres de Maven dans `Settings > Build, Execution, Deployment > Build Tools > Maven`.

Création d’un projet Maven simple



Créer un projet Maven avec un IDE

La création d'un projet Maven est facilitée dans les IDE modernes :

Avec IntelliJ IDEA :

- `File > New > Project`
- Choisir **Maven** comme type de projet
- Spécifier `groupId`, `artifactId`, `version`
- Structure du projet générée automatiquement

Avec Eclipse :

- `File > New > Maven Project`
- Suivre l'assistant de création
- Possibilité de choisir un **archetype** préconfiguré

Créer un projet Maven en ligne de commande

Il est possible de créer un projet Maven sans IDE grâce à la commande suivante :

```
mvn archetype:generate
```

Cette commande propose plusieurs **archetypes** (modèles) de projet. L'utilisateur peut :

- filtrer les templates
- choisir l'ID de l'archetype
- saisir le `groupId`, `artifactId`, `version`

Exemple : générer un projet simple `Hello Maven` avec l'archetype par défaut.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Exemple : petit projet "Hello Maven"

Arborescence générée :

```
hello-maven/
├── pom.xml
└── src/
    └── main/java/com/example/App.java
```

Code minimal dans `App.java` :

```java
package com.example;

public class App {
    public static void main(String[] args) {
        System.out.println("Hello Maven!");
    }
}
```

Compilation et exécution :

```
mvn compile
java -cp target/hello-maven-1.0-SNAPSHOT.jar com.example.App
```

Utiliser des dépendances externes



Rechercher une bibliothèque sur mvnrepository.com

Pour ajouter une dépendance externe, il est courant d'utiliser le site : 👉 https://mvnrepository.com

Ce site permet de :

- Rechercher une bibliothèque par nom ou mot-clé
- Consulter les dernières versions disponibles
- Copier la balise `<dependency>` prête à être insérée dans le `pom.xml`

Exemple : rechercher `Gson` ou `JUnit`.

Ajouter une bibliothèque externe au projet

Voici un exemple d'ajout de **Gson** pour parser du JSON :

```
<dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.10.1</version>
</dependency>
```

Placer ce bloc dans la section `<dependencies>` du fichier `pom.xml`. Maven télécharge automatiquement cette bibliothèque au prochain `mvn install` ou `mvn compile`.

Démo : parser du JSON avec Gson

Exemple de code :

```
import com.google.gson.Gson;

class User {
    String name;
    int age;
}

public class App {
    public static void main(String[] args) {
        String json = "{\"name\":\"Alice\",\"age\":30}";
        Gson gson = new Gson();
        User userEntity = gson.fromJson(json, User.class);
        System.out.println(userEntity.name + " - " + userEntity.age);
    }
}
```

Ce programme transforme une chaîne JSON en objet Java grâce à Gson.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Démo : test unitaire avec JUnit 5

Exemple de test :

```java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculatorTest {

    @Test
    void additionFonctionne() {
        assertEquals(4, 2 + 2);
    }
}
```

Ce test est automatiquement détecté et exécuté par Maven via `mvn test`.

Ajouter une dépendance dans IntelliJ IDEA

IntelliJ propose plusieurs moyens pour gérer les dépendances Maven :

- Cliquer sur le fichier `pom.xml`, puis utiliser le panneau *Maven* pour synchroniser les dépendances.
- IntelliJ détecte automatiquement les nouvelles balises `<dependency>` ajoutées manuellement.
- Une alerte "Import Changes" permet de recharger le projet.
- Possibilité de rechercher et ajouter une dépendance via `Alt + Insert` (ou clic droit > *Add Maven Dependency*).

💡 IntelliJ télécharge automatiquement les fichiers nécessaires depuis les dépôts distants.

Avantages et bonnes pratiques



Réutilisabilité des modules

Maven permet de découper une application en **plusieurs modules** :

- Chaque module a son propre `pom.xml`
- Un `pom` parent gère les versions partagées et dépendances communes
- Les modules peuvent être utilisés de façon autonome ou groupée

Cela favorise la **mutualisation** du code dans les architectures complexes.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Versionnage centralisé des dépendances

Avec Maven, il est possible de :

- Centraliser les versions des dépendances dans un `pom` parent (ou avec `<dependencyManagement>`)
- Réduire les conflits entre modules
- Faciliter la mise à jour globale d'une librairie

💡 C'est particulièrement utile dans les projets multi-modules ou les grandes équipes.

Builds reproductibles et intégration continue

Maven garantit des **builds reproductibles** car :

- Toutes les dépendances sont définies dans le `pom.xml`
- Le cycle de vie est standardisé et automatisé

Il s'intègre naturellement dans les pipelines CI/CD (Jenkins, GitLab CI, etc.) :

- Exécution des tests
- Packaging
- Déploiement automatisé

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Bonnes pratiques recommandées

Voici quelques règles clés :

- ✅ **Éviter les versions SNAPSHOT** en production
- ✅ **Versionner le fichier** `pom.xml` dans le dépôt Git
- ✅ **Vérifier les dépendances transitives** avec `mvn dependency:tree`
- ✅ **Nettoyer régulièrement** avec `mvn clean`
- ✅ **Documenter** les plugins et propriétés personnalisées

Ces pratiques améliorent la fiabilité et la maintenabilité du projet.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**


Pour aller plus loin

Maven multi-modules

Un projet **multi-modules** permet de gérer plusieurs sous-projets avec un seul `pom` parent :

Exemple d'arborescence :

```
mon-projet/
├── pom.xml (parent)
├── module-core/
│   └── pom.xml
└── module-web/
    └── pom.xml
```

Avantages :

- Meilleure séparation des responsabilités
- Compilation et tests modulaires
- Réutilisation de code entre modules

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Intégration continue avec Maven

Maven s'intègre très facilement avec les outils de CI :

- **Jenkins** : plugin Maven intégré
- **GitLab CI** / **GitHub Actions** : simple à configurer via `.gitlab-ci.yml` ou `.github/workflows`

Pipeline typique :

- `mvn clean install`
- `mvn test`
- `mvn deploy`

Permet l'automatisation de la qualité, des livraisons et du déploiement.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Introduction aux profils Maven

Les **profils Maven** permettent d'adapter la configuration selon l'environnement ou les besoins :

```
<profiles>
  <profile>
    <id>dev</id>
    <properties>
      <env.name>développement</env.name>
    </properties>
  </profile>
  <profile>
    <id>prod</id>
    <properties>
      <env.name>production</env.name>
    </properties>
  </profile>
</profiles>
```

Cela permet d'ajuster des comportements sans modifier le `pom.xml` principal.
Activation avec :

```
mvn clean install -Pdev
```

Liens utiles

🔗 **Documentation officielle Maven** : https://maven.apache.org

🔍 **Dépôt de dépendances Maven Central** : https://search.maven.org

📦 **Explorateur de dépendances** : https://mvnrepository.com

🧪 **JUnit 5 (tests)** : https://junit.org/junit5/

🐙 **Guide CI/CD avec Maven (GitHub Actions)** : https://docs.github.com/en/actions

🔄 **Guide CI/CD avec Maven (GitLab CI/CD)** : https://docs.gitlab.com/ci/

📘 **Tutoriels Maven - Baeldung** : https://www.baeldung.com/maven

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

🤝 Merci de ton attention ! 🤝

Reste curieux 🧐 et amuse-toi ! 🎮 🎉

Ce cours t'a été présenté par

 Moi-même, Nathaniel Vaur Henel 🤘

 mon site de partage autour du dév : dev-sensei.digicrafters.fr

 mon Framagit perso, GitLab éthique, libre & français : framagit.org/nathvh/me

 mon GitHub perso, mais inactif : github.com/Nathaniel-Vaur-Henel

 mon LinkedIn, pour suivre mon parcours pro : linkedin.com/in/nathaniel-vaur-henel

 mon Discord pro, pour échanger ou me poser des questions : `@nathanielvaurhenel`